```
# Import Libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_squared_error, r2_score
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout
from tensorflow.keras.callbacks import EarlyStopping, ModelCheckpoint
!pip install keras-tuner
import keras_tuner as kt
```

```
Requirement already satisfied: keras-tuner in /usr/local/lib/python3.11/dist-packages (1.4.7)
Requirement already satisfied: keras in /usr/local/lib/python3.11/dist-packages (from keras-tuner) (3.8.0)
Requirement already satisfied: packaging in /usr/local/lib/python3.11/dist-packages (from keras-tuner) (24.2)
Requirement already satisfied: requests in /usr/local/lib/python3.11/dist-packages (from keras-tuner) (2.32.3)
Requirement already satisfied: kt-legacy in /usr/local/lib/python3.11/dist-packages (from keras-tuner) (1.0.5)
Requirement already satisfied: absl-py in /usr/local/lib/python3.11/dist-packages (from keras->keras-tuner) (1.4.0)
Requirement already satisfied: numpy in /usr/local/lib/python3.11/dist-packages (from keras->keras-tuner) (2.0.2)
Requirement already satisfied: rich in /usr/local/lib/python3.11/dist-packages (from keras->keras-tuner) (13.9.4)
Requirement already satisfied: namex in /usr/local/lib/python3.11/dist-packages (from keras->keras-tuner) (0.0.8)
Requirement already satisfied: h5py in /usr/local/lib/python3.11/dist-packages (from keras->keras-tuner) (3.13.0)
Requirement already satisfied: optree in /usr/local/lib/python3.11/dist-packages (from keras->keras-tuner) (0.15.0)
Requirement already satisfied: ml-dtypes in /usr/local/lib/python3.11/dist-packages (from keras->keras-tuner) (0.4.1)
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.11/dist-packages (from requests->keras-tuner) (3.4
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.11/dist-packages (from requests->keras-tuner) (3.10)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.11/dist-packages (from requests->keras-tuner) (2.3.0)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.11/dist-packages (from requests->keras-tuner) (2025.1.31
Requirement already satisfied: typing-extensions>=4.5.0 in /usr/local/lib/python3.11/dist-packages (from optree->keras->keras-tuner
Requirement already satisfied: markdown-it-py>=2.2.0 in /usr/local/lib/python3.11/dist-packages (from rich->keras->keras-tuner) (3.0
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in /usr/local/lib/python3.11/dist-packages (from rich->keras->keras-tuner) (2
Requirement already satisfied: mdurl~=0.1 in /usr/local/lib/python3.11/dist-packages (from markdown-it-py>=2.2.0->rich->keras->keras
```

```
# Load the Dataset
df = pd.read_csv('Life Expectancy Data.csv')
df.head()
```

	Country	Year	Status	Life expectancy	Adult Mortality	infant deaths	Alcohol	percentage expenditure	Hepatitis B	Measles	...	Polio	Total expenditure	Dip
0	Afghanistan	2015	Developing	65.0	263.0	62	0.01	71.279624	65.0	1154	...	6.0	8.16	
1	Afghanistan	2014	Developing	59.9	271.0	64	0.01	73.523582	62.0	492	...	58.0	8.18	
2	Afghanistan	2013	Developing	59.9	268.0	66	0.01	73.219243	64.0	430	...	62.0	8.13	
3	Afghanistan	2012	Developing	59.5	272.0	69	0.01	78.184215	67.0	2787	...	67.0	8.52	
4	Afghanistan	2011	Developing	59.2	275.0	71	0.01	7.097109	68.0	3013	...	68.0	7.87	

5 rows × 22 columns

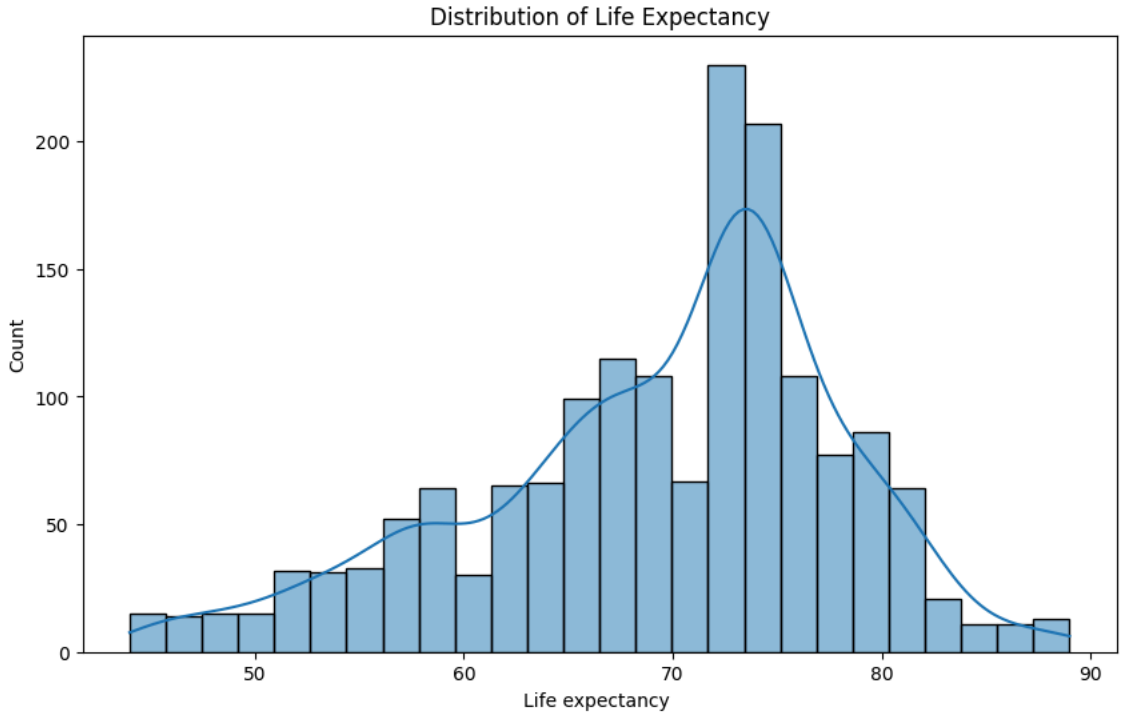
```
# EDA
print(df.info())
print(df.describe())
print(df.isnull().sum())
sns.heatmap(df.isnull(), cbar=False)
plt.title("Missing Values Heatmap")
plt.show()
```

Show hidden output

```
# Fill or Drop missing values
df = df.dropna()
df
```

Show hidden output

```
# Visualizations
plt.figure(figsize=(10, 6))
sns.histplot(df['Life expectancy '], kde=True)
plt.title("Distribution of Life Expectancy")
plt.show()
```




```
sns.pairplot(df[['Life expectancy ', 'Adult Mortality', 'infant deaths', 'GDP']])
plt.show()
```

Show hidden output

```
# Correlation heatmap
plt.figure(figsize=(15, 10))
# Select only numeric columns for correlation calculation
numerical_df = df.select_dtypes(include=np.number)
sns.heatmap(numerical_df.corr(), annot=True, fmt=".2f", cmap="coolwarm")
plt.title("Correlation Heatmap")
plt.show()
```

Show hidden output

```
#Check distributions and apply transformations if needed
from scipy.stats import skew

# Select only numeric columns for skewness calculation
numerical_df = df.select_dtypes(include=np.number)

# Calculate skewness for numeric columns
skewed_features = numerical_df.skew().sort_values(ascending=False)

print("Skewed features:\n", skewed_features)
```

```
Skewed features:
 Population                       14.186299
infant deaths                     8.477369
under-five deaths                 8.340863
Measles                           7.957838
percentage expenditure            4.980574
 HIV/AIDS                         4.974176
GDP                               4.517297
 thinness 5-9 years               1.866980
 thinness  1-19 years             1.821074
Adult Mortality                   1.276429
Alcohol                           0.662518
Total expenditure                 0.213362
Schooling                        -0.128164
Year                             -0.200171
 BMI                             -0.233601
Life expectancy                  -0.628758
Income composition of resources  -1.155244
Hepatitis B                      -1.793377
Polio                            -2.360177
Diphtheria                       -2.487492
dtype: float64
```

```
# Optional: Log transform skewed features
skewed_cols = skewed_features[skewed_features > 1].index
```

```
df[skewed_cols] = df[skewed_cols].apply(lambda x: np.log1p(x))


# Feature Scaling and Outlier Treatment
# (Outliers already partially treated via log)
X = df.drop(['Life expectancy ', 'Country', 'Year', 'Status'], axis=1) # Drop the 'Status' column
y = df['Life expectancy ']
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)


X_train, X_test, y_train, y_test = train_test_split(X_scaled, y, test_size=0.2, random_state=42)


#Build ANN Model

def build_model():
    model = Sequential([
        Dense(128, activation='relu', input_shape=(X_train.shape[1],)),
        Dropout(0.3),
        Dense(64, activation='relu'),
        Dropout(0.2),
        Dense(1)
    ])
    model.compile(optimizer='adam', loss='mse', metrics=['mae'])
    return model

model = build_model()
```

```
/usr/local/lib/python3.11/dist-packages/keras/src/layers/core/dense.py:87: UserWarning: Do not pass an `input_shape`/`input_dim` arg
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

```
# Callbacks
early_stop = EarlyStopping(monitor='val_loss', patience=10)
checkpoint = ModelCheckpoint('best_model.h5', save_best_only=True)



# Train Model
history = model.fit(
    X_train, y_train,
    validation_split=0.2,
    epochs=100,
    batch_size=32,
    callbacks=[early_stop, checkpoint],
    verbose=1
)
```

Show hidden output

```
model.load_weights('best_model.h5')
y_pred = model.predict(X_test)
print("MSE:", mean_squared_error(y_test, y_pred))
print("RMSE:", np.sqrt(mean_squared_error(y_test, y_pred)))
print("R2 Score:", r2_score(y_test, y_pred))
```

```
11/11 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step
MSE: 14.680074910300037
RMSE: 3.831458587835713
R2 Score: 0.793303838671104
```

```
# Hyperparameter Tuning with KerasTuner
def model_builder(hp):
    model = Sequential()
    model.add(Dense(units=hp.Int('units1', min_value=64, max_value=256, step=32), activation='relu'))
    model.add(Dropout(hp.Float('dropout1', 0.2, 0.5, step=0.1)))
    model.add(Dense(units=hp.Int('units2', min_value=32, max_value=128, step=16), activation='relu'))
    model.add(Dense(1))

    model.compile(
        optimizer=hp.Choice('optimizer', ['adam', 'rmsprop']),
        loss='mse',
        metrics=['mae']
    )
    return model



tuner = kt.RandomSearch(
    model_builder,
    objective='val_loss',
```

```
    max_trials=5,
    executions_per_trial=2,
    directory='my_dir',
    project_name='life_expectancy_tuning'
)

tuner.search(X_train, y_train, epochs=50, validation_split=0.2, callbacks=[early_stop])

best_hps = tuner.get_best_hyperparameters(num_trials=1)[0]
print(f"Best hyperparameters: {best_hps.values}")
```

```
Trial 5 Complete [00h 00m 35s]
val_loss: 12.818405628204346

Best val_loss So Far: 10.781323432922363
Total elapsed time: 00h 02m 53s
Best hyperparameters: {'units1': 192, 'dropout1': 0.30000000000000004, 'units2': 80, 'optimizer': 'rmsprop'}
```

```
# Final model with best hyperparameters
best_model = tuner.hypermodel.build(best_hps)
best_model.fit(X_train, y_train, validation_split=0.2, epochs=100, callbacks=[early_stop])
```

Show hidden output

```
# Final Evaluation
y_pred_final = best_model.predict(X_test)
print("Tuned RMSE:", np.sqrt(mean_squared_error(y_test, y_pred_final)))
print("Tuned R2 Score:", r2_score(y_test, y_pred_final))
```

```
11/11 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step
Tuned RMSE: 2.993967312293822
Tuned R2 Score: 0.873788697585484
```